

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	Garantizar integridad referencial, rendimiento de consultas y trazabilidad de métricas para tareas y KPIs, evitando SQL disperso y acoplamiento entre capas.
Orientación de la decisión	Usar ORM de Django como único punto de acceso a MySQL, con un modelo ER que soporte: - Project 1..* Task, Task N..N User via TaskAssignment, Task 1..* TimeLog. - Project 1..* KPI, KPI 1..* KPI_Measure, Task N..N KPI via TaskKPI. - Report (generado por/para proyecto) y Notification (referencias opcionales a Project/Task/KPI). Aplicar migraciones versionadas, índices por consultas típicas y políticas de borrado (CASCADE donde aplique).
Supuestos	- ER aprobado en SDS; volumen académico (datos moderados). - Consultas frecuentes: tareas por proyecto/estado/usuario; medidas de KPI por fecha; notificaciones no leídas.
Restricciones	MySQL como SGBD; tipos compatibles (DECIMAL para KPIs, JSON para filtros reportes). - Uso del ORM (no SQL ad-hoc desde UI/API). - Auditoría básica con timestamps.
Requerimientos relacionados	UC-01/02/03/04/05/12/18 (ciclo de vida de tareas), UC-06/09 (KPIs), UC-07/08 (dashboard/reportes), UC-15/16 (notificaciones), UC-10/11/13/14/17 (consultas/listas/progreso).
Alternativas consideradas	Alternativa 1: SQL directo desde vistas/controladores. Alternativa 2: Microservicios por bounded context (BD por servicio). Alternativa 3: ORM centralizado + ER normalizado (propuesta).
Evaluación de alternativas	Alternativa 1: SQL directo Ventajas: control fino y performance puntual. Desventajas: duplicación, riesgo de inconsistencias, difícil testing y mantenimiento. Alternativa 2: Microservicios + BD por servicio Ventajas: escalabilidad independiente. Desventajas: sobrecoste (operación/consistencia transaccional), no viable en el curso.

	Alternativa 3: ORM centralizado + ER • Ventajas: integridad, migraciones controladas, trazabilidad UC→tabla, reutilización de repositorios/servicios. • Desventajas: tuning puntual necesario para reportes pesados (se mitiga con índices y jobs asíncronos).
Alternativa seleccionada y Justificación	ORM de Django + ER normalizado (monolito modular). Define entidades: User, Project, Task, TaskAssignment, TimeLog, KPI, KPI_Measure, TaskKPI, Report, Notification. Asegura integridad para los flujos críticos (UC-01/02/04/06/08/15) y mantiene un único contrato de datos.
Decisiones relacionadas	• Políticas de borrado: Task CASCADE a TimeLog, TaskAssignment, TaskKPI, Notification; KPI CASCADE a KPI_Measure y TaskKPI. • Índices: Task(project_id,status,due_date), TimeLog(task_id,user_id,work_date), KPI_Measure(kpi_id,measure_date DESC), Notification(user_id,read_at). • Jobs asíncronos (Celery) para reportes y cálculos pesados de KPIs.
Comentarios	Buenas prácticas: unique (TaskKPI(task_id,kpi_id)), checks (TimeLog.minutes>0), FKs NULLABLE en Notification (referencias opcionales). Preparado para exportar a PDF/XLSX y cachear lecturas de dashboard.